TECHNICAL REFERENCE & CURRICULUM GUIDE

Module 9 (Part 1): Embeddings, Vector Databases & RAG Systems

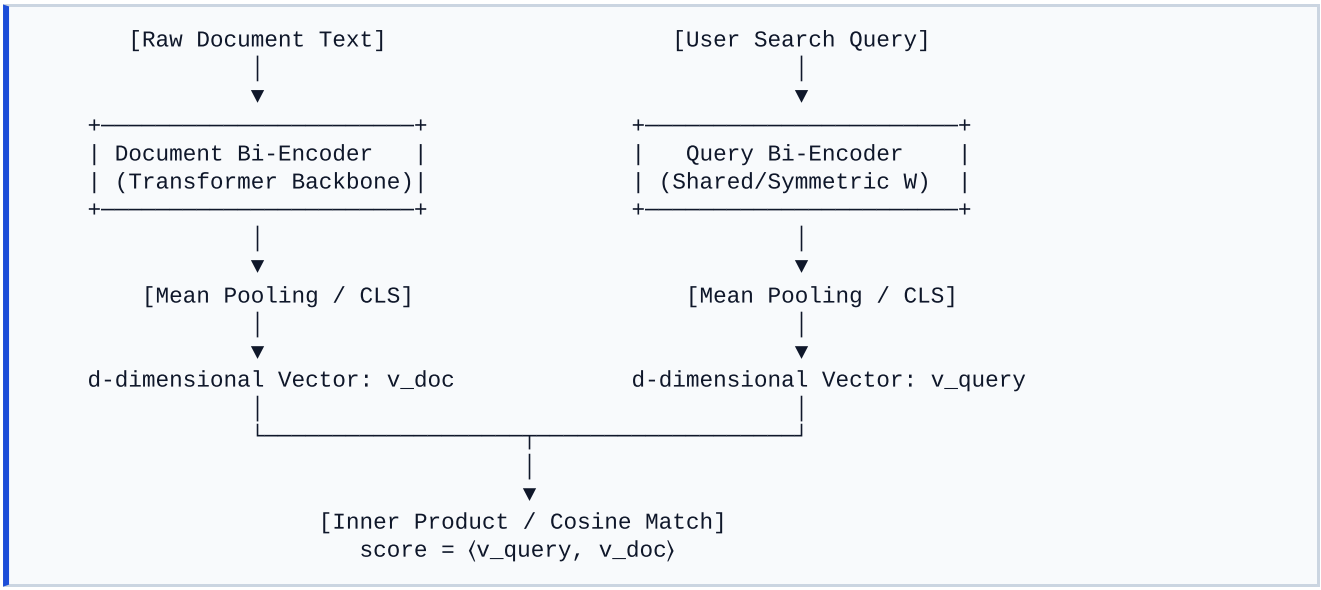
Module 9 Part 1 details the storage, mathematical modeling, and retrieval foundations of Retrieval-Augmented Generation (RAG): from dense embedding geometry and chunking algorithms to vector database internals, Approximate Nearest Neighbor (ANN) index structures, and multi-tenant architectures.

9.1 Dense Embedding Models: OpenAI, Cohere, and Sentence-Transformers

A **dense embedding model** maps unstructured text sequences of arbitrary token length T into a fixed-dimensional, continuous vector space:

$$f_\theta : \mathcal{V}^T \longrightarrow \mathbb{R}^d$$

Unlike traditional sparse representations (such as One-Hot encodings or high-dimensional TF-IDF bags of words with dimensionalities exceeding $|V| \geq 100{,}000$), dense embeddings project semantic concepts into lower-dimensional manifolds (typically $d \in [384, 3072]$). In this space, semantic similarity correlates with geometric proximity.



1. The Bi-Encoder Architecture and Contrastive Pre-Training

Modern dense embeddings are produced by **Bi-Encoder** Transformer architectures trained via **Contrastive Learning**. Given a batch of B query-document pairs $\{(q_i, p_i^+)\}_{i=1}^{B}$ where p_i^+ is the ground-truth positive passage for query q_i, the other $B-1$ passages in the batch serve as **in-batch negative samples** $p_{i,j}^-$.

The model minimizes the symmetric **InfoNCE (Information Noise-Contrastive Estimation) Loss**:

$$L_{\text{InfoNCE}} = -\sum_{i=1}^{B} \log \frac{\exp(\text{sim}(q_i, p_i^+) / \tau)}{\exp(\text{sim}(q_i, p_i^+) / \tau) + \sum_{j \neq i} \exp(\text{sim}(q_i, p_j^+) / \tau)}$$

where:

- $\text{sim}(u, v) = (u \cdot v) / (\|u\|_2 \|v\|_2)$ is the cosine similarity.
- τ is a learnable or fixed temperature parameter (typically $\tau \in [0.01, 0.07]$) that sharpens the Softmax distribution over hard negatives.

Pooling Mechanics: Converting Token Sequences to a Single Vector

Because an Encoder outputs a tensor of shape (B, T, d_{model}), the sequence dimension T must be collapsed into a single vector $\mathbb{R}^d$:

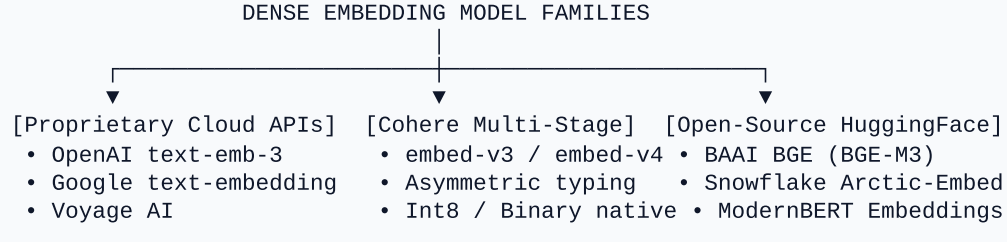
- `[CLS]` **Token Pooling:** Takes the first token representation $h_0 \in \mathbb{R}^{d_{model}}$. Standard in original BERT, though susceptible to token representation collapse without contrastive fine-tuning.
- **Mean Pooling (Global Average Pooling):** Computes the element-wise average across all non-padding token hidden states:

$$v = \frac{\sum_{t=1}^{T} m_t \cdot h_t}{\sum_{t=1}^{T} m_t}$$

where $m_t \in \{0, 1\}$ is the attention mask bit. Mean pooling preserves sentence-wide semantic context better than `[CLS]` across transfer tasks.

- **Last-Token Pooling:** Used by causal Decoder-only embedding models (e.g., embedding variants of Mistral, Qwen, or LLaMA), where the final token has causally attended to all preceding tokens in the sequence.

2. Major Dense Embedding Model Families



OpenAI `text-embedding-3` Family

- **Architectures:** `text-embedding-3-small` ($d = 1536$) and `text-embedding-3-large` ($d = 3072$).
- **Maximum Context Length:** 8,191 input tokens.
- **Core Technological Feature:** Native **Matryoshka Representation Learning (MRL)**. Allows developers to shorten vectors (e.g., truncating `3-large` from 3072 down to 1024, 512, or 256 dimensions) via API parameter `dimensions=512` without retraining, reducing storage footprints while retaining retrieval recall.

Cohere `embed-v3` / `embed-v4` Family

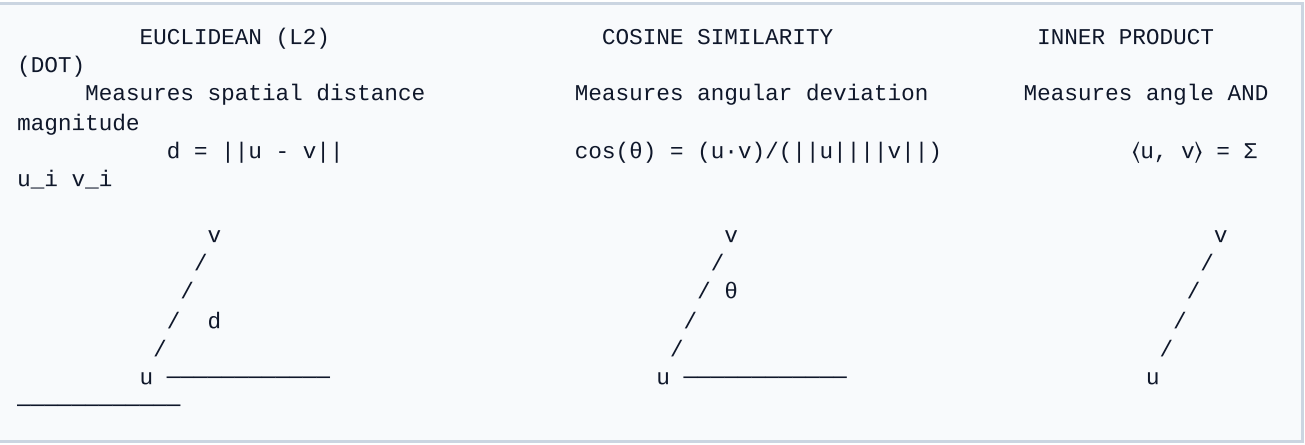
- **Architectures:** Multilingual and English-only models ($d = 1024$).
- **Task-Specific Conditioning (`input_type`):** Cohere enforces asymmetric bi-encoder conditioning by requiring developers to classify input vectors:
  - `input_type="search_document"`: Ingests documents into the vector store.
  - `input_type="search_query"`: Formats incoming user search strings.
  - `input_type="classification"` or `"clustering"`: Adjusts embedding geometry for unsupervised grouping.
- **Native Quantization Embeddings:** Supports direct emission of compressed embedding representations (`float`, `int8`, `uint8`, `binary`, `ubinary`), eliminating downstream quantization steps.

Open-Source / Hugging Face Ecosystem (`sentence-transformers`)

- **BAAI BGE (`BGE-M3`, `BGE-large-en-v1.5`):** High-performing models on the Massive Text Embedding Benchmark (MTEB). `BGE-M3` supports **Multi-Linguality** (100+ languages), **Multi-Granularity** (up to 8,192 tokens), and **Multi-Functionality** (simultaneously emitting dense embeddings, sparse lexical weights like SPLADE, and multi-vector ColBERT-style token representations from a single forward pass).
- **Snowflake `Arctic-Embed` (v2.0):** Enterprise models trained with multi-task contrastive pipelines, offering native MRL and 256-dimension binary-quantized modes.
- **ModernBERT / Nomic-Embed:** Transformer encoders featuring unpadded flash attention, rotary positional embeddings (RoPE), and 8,192 token context windows natively running on local hardware.

9.2 Similarity Metrics: Cosine, Dot Product, Euclidean

The mathematical metric chosen to evaluate proximity between a query vector $\boldsymbol{q}$ and a document vector $\boldsymbol{d}$ governs the geometric manifold, query latency, and index efficiency.



1. Mathematical Formulations

Euclidean Distance (L_2 Distance)

Measures the straight-line metric distance between two coordinates in $\mathbb{R}^d$:

$$D_{L_2}(\mathbf{q}, \mathbf{d}) = \|\mathbf{q} - \mathbf{d}\|_2 = \sqrt{\left(\sum_{i=1}^{d} (q_i - d_i)^2 \right)}$$

- **Range:** $[0, \infty)$. A distance of 0.0 signifies identical vector coordinates.
- **Sensitivity:** Highly sensitive to differences in absolute vector magnitude $\|\boldsymbol{x}\|_2$.

Dot Product (Inner Product / IP)

Measures the projection of one vector along the axis of another, scaling simultaneously with angle and length:

$$\langle \mathbf{q}, \mathbf{d} \rangle = \mathbf{q} \cdot \mathbf{d} = \sum_{i=1}^{d} q_i \, d_i = \|\mathbf{q}\|_2 \, \|\mathbf{d}\|_2 \cos(\theta)$$

- **Range:** $(-\infty, \infty)$.
- **Sensitivity:** Heavily favors vectors with larger norms. In search systems where documents vary widely in text volume or token counts, unnormalized dot product skews results toward longer documents.

Cosine Similarity

Measures the angular difference between two directional vectors, normalizing out length:

$$S_{\cos}(\mathbf{q}, \mathbf{d}) = \cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\|_2 \, \|\mathbf{d}\|_2} = \frac{\sum_{i=1}^{d} q_i \, d_i}{\sqrt{\left(\sum_{i=1}^{d} q_i^2\right)} \sqrt{\left(\sum_{i=1}^{d} d_i^2\right)}}$$

- **Range:** *[−1.0, 1.0]*. A value of *1.0* indicates identical direction ($\theta = 0°$); *0.0* indicates orthogonality ($\theta = 90°$); *−1.0* indicates diametrically opposing directions ($\theta = 180°$).
- **Cosine Distance:** Often calculated as $D_{cos} = 1 - S_{cos}$.

2. The L_2 Normalization Equivalence Theorem

In modern production pipelines, vector databases rarely compute the full Cosine Similarity formula at query time due to the computational overhead of calculating square roots in the denominator. Instead, all vectors are ***L_2*-normalized to unit length** upon ingestion:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2} \implies \|\hat{\mathbf{v}}\|_2 = 1.0$$

When $\|\hat{\boldsymbol{q}}\|_2 = 1$ and $\|\hat{\boldsymbol{d}}\|_2 = 1$:

1. **Cosine Similarity collapses into Dot Product:**

$$S_{\cos}(\hat{\mathbf{q}}, \hat{\mathbf{d}}) = \frac{\hat{\mathbf{q}} \cdot \hat{\mathbf{d}}}{(1.0)(1.0)} = \hat{\mathbf{q}} \cdot \hat{\mathbf{d}}$$

2. **Euclidean Distance maps monotonically to Dot Product:**

$$\|\hat{\mathbf{q}} - \hat{\mathbf{d}}\|_2^2 = \|\hat{\mathbf{q}}\|_2^2 + \|\hat{\mathbf{d}}\|_2^2 - 2(\hat{\mathbf{q}} \cdot \hat{\mathbf{d}}) = 1 + 1 - 2(\hat{\mathbf{q}} \cdot \hat{\mathbf{d}}) = 2(1 - \hat{\mathbf{q}} \cdot \hat{\mathbf{d}}) \implies \|\hat{\mathbf{q}} - \hat{\mathbf{d}}\|_2 = \sqrt{2(1 - S_{\cos})}$$

- **Production Implication:** If embeddings are unit-normalized upon creation, **minimizing Euclidean Distance, maximizing Dot Product, and maximizing Cosine Similarity yield identical nearest-neighbor rankings**. Therefore, production systems store unit-normalized vectors and configure their indexes for **Inner Product (IP)**, executing only fused multiply-accumulate (FMA) instructions via hardware SIMD registers without square roots or runtime division.

Metric	Metric Space Invariant	Hardware Cost (FLOPs)	Optimal Production Use Case
Cosine	Invariant to vector scale / norm	High (d mults, $2d$ squares, 2 sqrt, 1 div)	Unnormalized text representations, raw frequency profiles
Dot Product (IP)	Scaled by both angle and norm	Lowest (d multiply-accumulates via SIMD)	**Normalized embeddings (Default production configuration)**
Euclidean (L_2)	Sensitive to absolute coordinate distance	Moderate (d subtractions, d squares, 1 sqrt)	Clustering (k-means), image coordinate embeddings, spatial data

9.3 Chunking Strategies: Algorithmic Implementations

The chunking layer establishes the atomic retrieval resolution for a RAG architecture.

- **The Granularity Dilemma:**
  - **Chunks too small (e.g., 50 tokens):** Context is fragmented. Key pronouns lose their antecedents, leading to isolated vectors that match queries out of context.
  - **Chunks too large (e.g., 2,000 tokens):** The embedding vector washes out specific factual assertions, diluting dense retrieval precision and overloading the LLM's prompt budget with irrelevant noise.

```
1. Fixed-Size Overlapping Window:
[       Chunk 1: 500 chars       ]
               [       Chunk 2: 500 chars       ]
                              [       Chunk 3: 500 chars       ]

2. Recursive Character Splitting:
[Paragraph 1: Split on \n\n] -> If > Max -> Split on \n -> If > Max -> Split on space

3. Semantic Chunking (Cosine Distance Breakpoint):
[Sentence 1][Sentence 2][Sentence 3]   |BREAK|   [Sentence 4][Sentence 5]
 (Distance d12=0.08, d23=0.11)                   (Distance d34=0.62 -> New topic!)

4. Structure-Aware (Document Hierarchy):
# Module Header
## Section 2.1 [Header Context prepended]
| Markdown Table Kept As Atomic Unit |
```

1. Fixed-Size Sliding Window Chunking

Chops raw text strings at fixed character or token counts C, advancing by a stride $S = C - O$ (where O is the overlap).

- **Flaw:** Blind to grammatical boundaries. It cuts numbers, names, and sentences in half (e.g., splitting `"The loss was $40"` into Chunk 1: `"...The loss was $"` and Chunk 2: `"40 million..."`), introducing corruptions into downstream vector search.

2. Recursive Character Text Splitting

The production baseline for plain text (e.g., `RecursiveCharacterTextSplitter`). It traverses a prioritized list of linguistic separators:

> Separators = [`"\n\n"`, `"\n"`, `". "`, `" "`, `""`]

- **Mechanics:** It attempts to split at paragraph breaks (`\n\n`). If a resulting block exceeds the maximum chunk size, it recursively steps down to sentence breaks (`\n`, `. `), then word breaks (` `), only splitting mid-word as a last resort. This keeps paragraphs and complete thoughts structurally intact.

3. Semantic Chunking (Embedding Distance Breakpoints)

Instead of relying on arbitrary character lengths, **Semantic Chunking** uses an embedding model to locate shifts in topical continuity.

1. The source text is split into a list of complete sentences $\{s_1, s_2, \ldots, s_n\}$ using a rule-based tokenizer (e.g., NLTK, spaCy).
2. To smooth local variance, each sentence s_i is combined with adjacent sentences into a small sliding context window to generate a sentence embedding:

$$\mathbf{e}_i = \text{Embed}(s_{i-1} \circ s_i \circ s_{i+1})$$

3. The semantic distance between adjacent sentence pairs is calculated:

$$d_i = 1 - S_{\cos}(\mathbf{e}_i, \mathbf{e}_{i+1})$$

4. Breakpoints are placed wherever semantic distance exceeds a dynamic statistical threshold:

$$\text{Threshold} = \mu_d + k \cdot \sigma_d \quad \text{or} \quad \text{Percentile}(d, 90)$$

where μ_d is the mean distance across the document and σ_d is the standard deviation.

5. Sentences between breakpoints are bundled into cohesive semantic chunks.

```
import numpy as np
from typing import List

def semantic_chunking(
    sentences: List[str],
    embeddings: np.ndarray,
    threshold_percentile: float = 85.0
) -> List[str]:
    """
    Groups sentences into variable-sized chunks based on semantic distance spikes.
    """
    # 1. Calculate cosine distance between adjacent sentence vectors
    # Assumes embeddings are L2-normalized
    dot_products = np.sum(embeddings[:-1] * embeddings[1:], axis=1)
    distances = 1.0 - dot_products

    # 2. Compute dynamic breakpoint threshold
    breakpoint_threshold = np.percentile(distances, threshold_percentile)

    # 3. Form chunks by splitting across detected boundaries
    chunks = []
    current_chunk = [sentences[0]]

    for i, dist in enumerate(distances):
        if dist > breakpoint_threshold:
            # Semantic topic shift detected: complete current chunk
            chunks.append(" ".join(current_chunk))
            current_chunk = [sentences[i + 1]]
        else:
            current_chunk.append(sentences[i + 1])

    if current_chunk:
        chunks.append(" ".join(current_chunk))

    return chunks
```

4. Structure-Aware / Document-Hierarchy Chunking

Modern enterprise RAG systems ingest structured inputs (Markdown, HTML, DOCX, OpenAPI specifications, SEC filings) using structural parsers.

- **Header Path Inheritance:** A chunk located three levels deep inherits its breadcrumb hierarchy:

  ```
  # AWS Enterprise VPC Architecture
  ## Network Access Control Lists (NACL)
  ### Inbound Ephemeral Rules
  [Body content of chunk: Port range 1024-65535 rules...]
  ```

  The text embedded is prepended with its metadata lineage: `AWS Enterprise VPC Architecture > Network Access Control Lists (NACL) > Inbound Ephemeral Rules: Port range 1024-65535 rules...`. This grounds the chunk's vector in the appropriate technical context.
- **Atomic Table Integrity:** Financial and tabular data is kept intact. Chunks are never split mid-row. Tables are converted into clean Markdown tables or serialized JSON key-value records to preserve relational links across rows and columns.

9.4 Embedding Dimensionality, Memory Footprints, and Matryoshka Embeddings (MRL)

Managing billions of high-dimensional floating-point vectors requires substantial memory resources.

1. Vector Memory & Storage Formulations

A collection of N vectors of dimensionality d stored in standard IEEE 754 Single-Precision Floating-Point (**FP32**) format consumes:

$$\text{Memory (Bytes)} = N \times d \times 4 \text{ bytes}$$

If using Half-Precision (**FP16 / BF16**):

$$\text{Memory (Bytes)} = N \times d \times 2 \text{ bytes}$$

Worked Production Scale Calculation (10 Million Vectors)

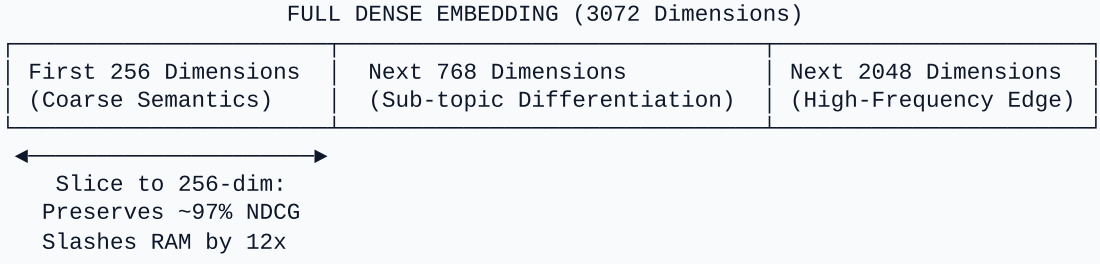
Suppose an enterprise indexes *10,000,000* chunks using `text-embedding-3-large` ($d = 3072$) in FP32:

1. **Raw Vector Memory:**
   $$10^7 \times 3072 \times 4 \text{ bytes} = 122{,}880{,}000{,}000 \text{ bytes} \approx \mathbf{122.88\ GB}$$
2. **Graph Index Overhead (HNSW Index):** Approximate Nearest Neighbor graph structures require storing edge adjacency lists, adding an extra 1.5× to 2.0× memory overhead:
   $$\text{Total Production RAM Required} \approx 122.88\ \text{GB} \times 1.75 \approx \mathbf{215\ GB\ of\ pure\ RAM}$$

Maintaining a 215 GB high-performance in-memory index on cloud instances (e.g., AWS `r6i.8xlarge`) carries high infrastructure expenses.

2. Matryoshka Representation Learning (MRL)

Introduced by Kusupati et al. (NeurIPS 2022), **Matryoshka Representation Learning** solves the memory-vs-accuracy trade-off by training a model to output vectors that behave like Russian nesting dolls.



- **Multi-Scale Loss Formulation:** Instead of optimizing loss only on the full-length vector d, MRL calculates the contrastive loss concurrently across a set of nested sub-vector prefix lengths $\mathcal{M} = \{m_1, m_2, ..., m_K\}$ (e.g., $\mathcal{M} = \{64, 128, 256, 512, 1024, 2048, 3072\}$):

  $$L_{\text{MRL}} = \sum_{m \in \mathcal{M}} w_m \cdot L_{\text{InfoNCE}}(\mathbf{q}_{1:m}, \mathbf{p}_{1:m}^{+})$$

  where $\boldsymbol{v}_{1:m}$ denotes taking only the first m indices of vector $\boldsymbol{v}$.

- **The Semantic Front-Loading Mechanism:** Backpropagating through this combined loss forces the network to concentrate the primary semantic signal into the **earliest dimensions**. The later dimensions are left to encode nuanced, fine-grained details.

3. Slicing, Normalizing, and Two-Stage Coarse-to-Fine Search

To use an MRL-enabled embedding at a reduced dimensionality, you slice the initial prefix and re-normalize it to unit length:

```
import numpy as np

def truncate_mrl_embedding(full_vector: np.ndarray, target_dim: int = 512) -> np.ndarray:
    # 1. Extract prefix slice
    sub_vector = full_vector[:target_dim]
    # 2. Re-apply L2 normalization to project back onto unit hypersphere
    norm = np.linalg.norm(sub_vector)
    if norm == 0:
        return sub_vector
    return sub_vector / norm
```

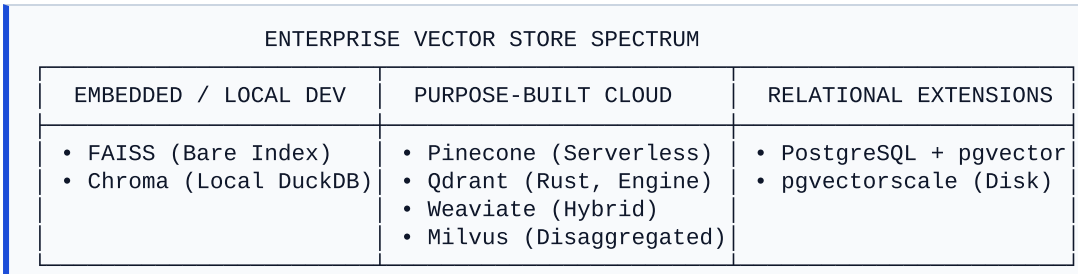
Production Architecture: Two-Stage Coarse-to-Fine Retrieval

MRL enables high-efficiency two-stage search over massive document corpora:

1. **Stage 1 (Coarse Fast Search):** Store the 256-dimensional truncated vectors in an in-memory index. A query retrieves the top $K = 500$ candidate document IDs in milliseconds, consuming 8% of the standard memory footprint.
2. **Stage 2 (Fine Rerank):** Fetch the full 3072-dimensional vectors from disk or an object store for only those 500 candidates, recalculate dot products, and return the final top 10 results.

9.5 The Vector Database Landscape: Comparative Analysis

A standard vector index (like FAISS) is an in-memory algorithm library, while a **Vector Database** is a full Data Management System providing ACID/concurrency guarantees, persistent disk-backed storage, live CRUD mutations, metadata filtering, horizontal sharding, role-based access control (RBAC), and point-in-time recovery.



1. Vector Database Architecture Matrix

Database Engine	Primary Language	Index Types Supported	Hybrid Search Support	Deployment Model	Sweet Spot & Operational Profile
FAISS	C++ / Python	Flat, IVF, HNSW, PQ, SQ	None (Bare vector index)	Embedded Library	High-performance offline batch indexing, GPU-accelerated nearest neighbor search.
Chroma	Python / Rust	HNSW, Brute-Force	Limited	Embedded / Client-Server	Local development, unit testing, prototyping, lightweight desktop workflows.
Pinecone	Proprietary	Managed Proprietary (HNSW/PQ variants)	Native (Dense + Sparse SPLADE)	Fully Managed Cloud (Serverless)	Zero index operations, automated scaling, serverless pay-per-read/write unit pricing.
Qdrant	Rust	HNSW, Inverted Index	Native (Dense + Sparse)	Self-Hosted / Managed Cloud	Rust engine, payload metadata filtering, memory-mapped disk storage.
Weaviate	Go	Dynamic HNSW, Flat	Built-in (BM25 + Dense vector fusion)	Self-Hosted (K8s) / Managed Cloud	Rich GraphQL/REST API, integrated vectorization modules, hybrid search.
Milvus	Go / C++	HNSW, IVF, SCANN, DiskANN, GPU	Native multi-vector & sparse	Distributed K8s / Zilliz Cloud	Enterprise-scale distributed architecture ($100\text{M}+$ to billions of vectors) with disaggregated compute and storage.
pgvector	C (Postgres ext)	HNSW, IVFFlat	Native (Postgres Full-Text Search tsvector + Dense)	Embedded in PostgreSQL	**Relational integration:** Enables single ACID queries joining vectors with relational business tables.

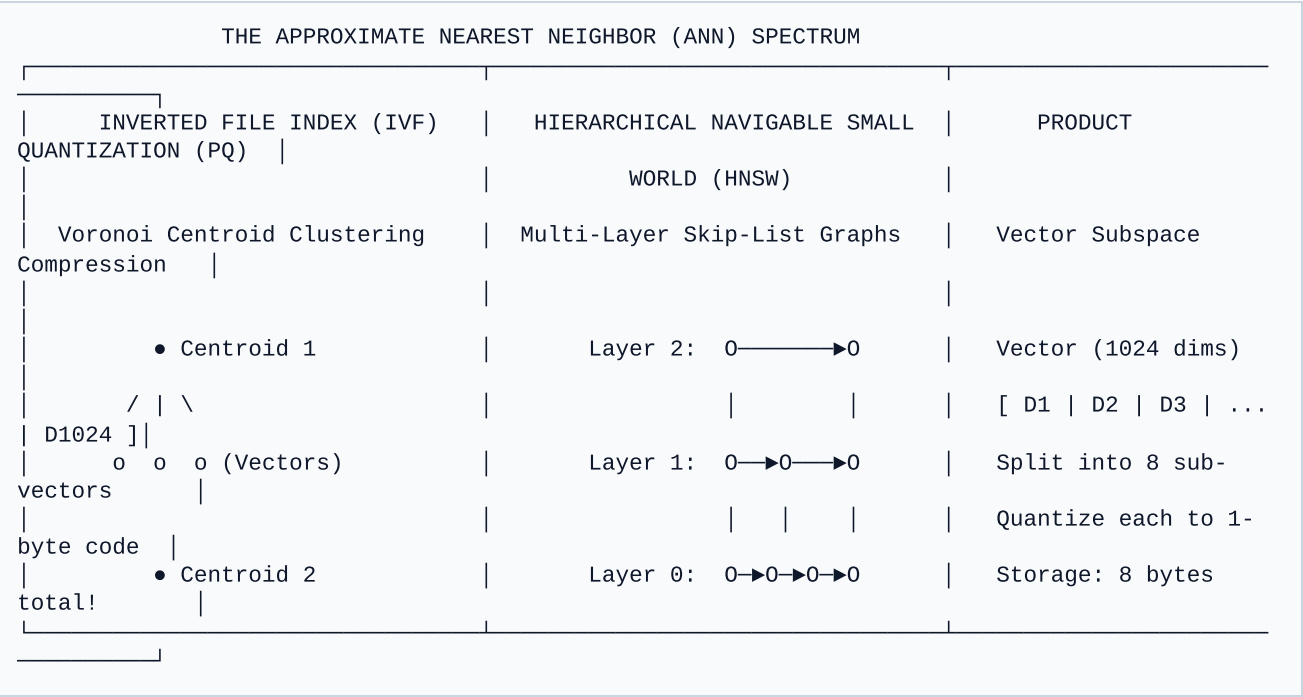
2. Choosing Between Dedicated Vector DBs vs. `pgvector`

- **Use `pgvector` when:** Your application already relies on PostgreSQL and holds fewer than 10–50 million vectors. It eliminates distributed sync issues, simplifies transactions, and allows filtering vectors directly against complex SQL `WHERE` clauses with relational foreign keys.
- **Use a Dedicated Vector DB (Qdrant, Milvus, Pinecone) when:** You exceed 50–100 million vectors, require high concurrent query throughput (*>1,000* QPS), need specialized sub-millisecond p99 latencies, or require advanced disaggregated storage layers (e.g., DiskANN).

9.6 Vector Indexing Algorithms: HNSW, IVF, and Product Quantization (PQ)

Brute-force Exact Nearest Neighbor search (Flat Indexing) compares a query against every single vector in the database (*O(N · d)* complexity). For a database of 50 million records, an exhaustive linear scan takes hundreds of milliseconds, making real-time search impractical.

Approximate Nearest Neighbor (ANN) algorithms trade a small margin of recall (e.g., retrieving 97% of true nearest neighbors instead of 100%) for logarithmic *O(\log N)* or constant *O(1)* query latencies.



1. Inverted File Index (IVF)

- **Algorithmic Structure:** IVF partitions the continuous vector space into Voronoi cells using **k-means clustering** to compute *C* centroids during an offline training phase. Each vector in the database is assigned to its nearest centroid, producing an **Inverted List** (posting list) per cell.

- **Query Phase:**
  1. The query vector $\boldsymbol{q}$ is compared against all C centroids ($O(C \cdot d)$ operations).
  2. The algorithm selects the n_{probe} closest centroids.
  3. It only scans the vectors located inside the posting lists of those n_{probe} clusters, bypassing the rest of the database.
- **Operational Hyperparameters:**
  - `nlist`: Number of cluster centroids generated during training.
  - `nprobe`: Number of neighboring centroids inspected at query time. Increasing `nprobe` improves recall at the expense of query latency.

2. Hierarchical Navigable Small World (HNSW) Graphs

HNSW (Malkov & Yashunin, 2018) is the standard index in production vector search engines due to its strong recall-vs-latency balance.

- **Algorithmic Structure:** HNSW builds a multi-layer graph based on the **Skip-List** principle.
  - **Top Layers:** Contain sparse graphs with long-range edges connecting distant points in the vector space.
  - **Bottom Layers:** Connectivity increases exponentially down to **Layer 0**, which forms a dense graph containing every node with local, short-range edges.
- **Search Mechanics:**
  1. Search begins at a fixed entry point in the highest layer.
  2. Greedy routing: The algorithm navigates to neighbor nodes that minimize distance to query $\boldsymbol{q}$.
  3. When a local minimum is reached in the current layer, the search drops down to the corresponding node in the layer below.
  4. This routing repeats down to **Layer 0**, where a local search explores candidates within a dynamic priority queue of size *efSearch* to locate the nearest neighbors.
- **Key Hyperparameters:**
  - M (typically 16–64): Maximum number of bi-directional connection links per node. Higher M improves recall on complex high-dimensional datasets but increases memory footprint.
  - *efConstruction* (typically 100–400): Search depth explored while assembling edges during index building. Higher values slow down indexing but improve the final graph structure.
  - *efSearch* (typically 32–256): Size of the dynamic candidate list evaluated during query execution. Governs the runtime latency-vs-recall trade-off.

3. Vector Quantization: Scalar (SQ) vs. Product Quantization (PQ)

Quantization compresses high-dimensional floating-point vectors into compact byte codes, fitting indexes into RAM and accelerating distance comparisons.

Scalar Quantization (SQ8)

Maps each 32-bit float coordinate independently into an 8-bit unsigned integer (*[0, 255]*) using linear scaling:

$$x_{\text{int8}} = \text{round}\left(255 \times \frac{x_{\text{float}} - \min_d}{\max_d - \min_d} \right)$$

- **Compression Ratio:** Exactly **4× memory reduction** (from 4 bytes down to 1 byte per dimension) with minimal recall degradation (*<1%*).

Product Quantization (PQ)

Compresses vectors through subspace decomposition:

1. A d-dimensional vector is split into m orthogonal sub-vectors, each of dimension $d^* = d / m$ (e.g., a 1024-dimensional vector split into $m = 64$ sub-vectors of size 16).
2. For each of the m subspaces, k-means clustering runs independently on training data to learn $k^* = 256$ centroids.
3. Each sub-vector is replaced by the **1-byte index** (*0 … 255*) of its nearest centroid.
4. **Compression Ratio:** A 1024-dimensional vector (4096 bytes in FP32) is compressed into **64 bytes**, achieving an **exhaustive 64× memory reduction**.
5. **Asymmetric Distance Computation (ADC):** At query time, the raw uncompressed query $\boldsymbol{q}$ is projected against the precomputed centroid codebooks, generating an $m \times 256$ distance lookup table. Distance calculations between the query and billions of compressed vectors are resolved using lookup table lookups and byte additions, without floating-point matrix multiplication.

9.7 Metadata Filtering, Hybrid Indexes, and Multi-Tenancy

Real-world production queries rarely search unstructured vectors in isolation; they enforce metadata constraints (e.g., *"Find documents semantically similar to 'credit facility' WHERE tenant_id = 'acme' AND year >= 2025"*).

METADATA FILTERING PARADIGMS

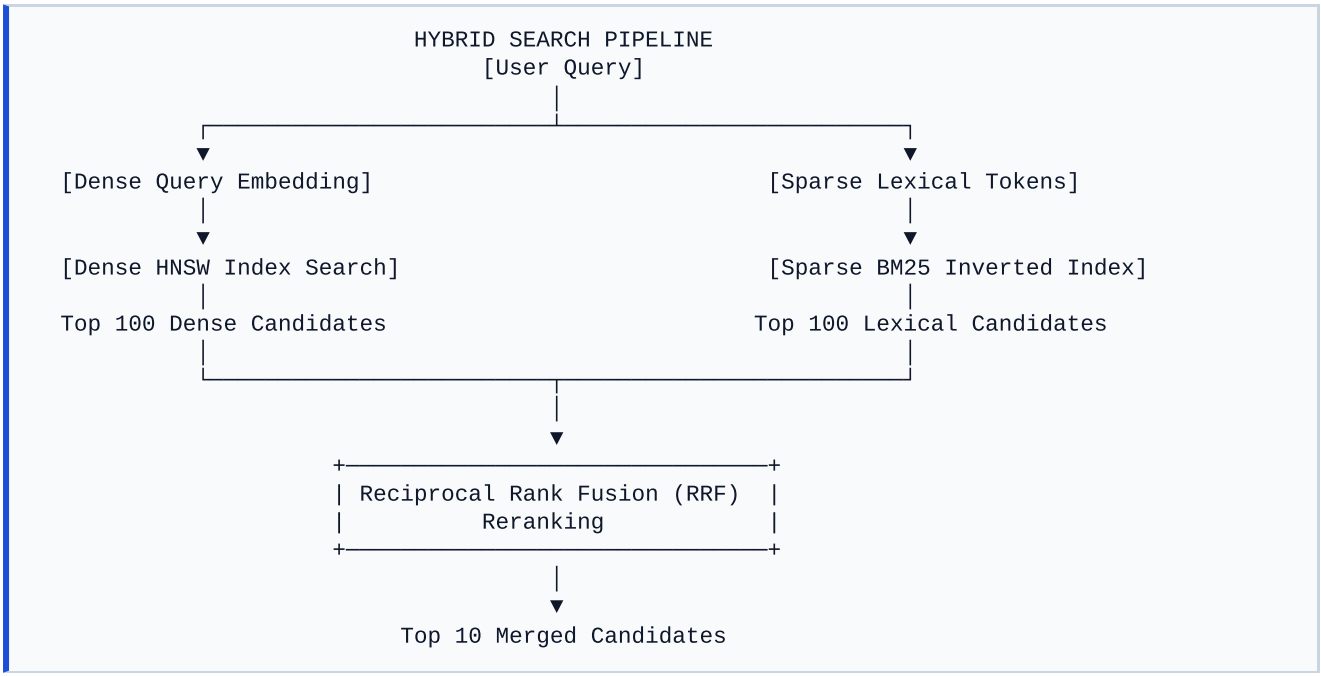
PRE-FILTERING	POST-FILTERING	SINGLE-STAGE HNSW
1. Filter relational DB 2. Brute-force vectors in filtered subset Flaw: Slow on broad metadata conditions.	1. Standard HNSW Search 2. Discard mismatches from top-K Flaw: Catastrophic Recall Collapse if filter is selective.	1. Graph traversal inspects metadata bitmask at each edge Optimal: Preserves both recall and execution speed.

1. Metadata Filtering Paradigms: Pre, Post, and Filtered HNSW

- **Naive Pre-Filtering:** Resolves the metadata predicate first using a relational/inverted index to obtain matching document IDs, then performs brute-force vector search over that isolated subset.
  - *Failure Mode:* If the filter matches 2 million records, running a brute-force scan across that subset degrades query throughput.
- **Naive Post-Filtering:** Runs standard HNSW search across the entire index to retrieve the top K nearest vectors (e.g., $K = 50$), then evaluates metadata conditions to prune non-matching items.
  - *Failure Mode (Catastrophic Recall Collapse):* If only 0.1% of vectors in the database belong to `tenant_id = 'acme'`, all 50 top vector neighbors may belong to other tenants. Pruning them leaves the user with **0 returned results**, despite thousands of valid matching documents existing elsewhere in the index.
- **Single-Stage Filtered Graph Traversal (The Production Standard):** Engines like Qdrant, Milvus, and Weaviate weave payload checks into graph traversal. When exploring edges from node u to node v, the search checks a bitmask of matching document IDs. If node v fails the metadata predicate, its distance is not evaluated, but its edges can still serve as routing hops to navigate toward valid nodes deeper in the graph.

2. Hybrid Search: Combining Dense Semantic and Sparse Lexical Retrieval

Dense embeddings understand semantic context and conceptual synonyms, but can fail on exact token matches, specialized serial numbers, API paths, and rare acronyms. Hybrid search combines dense retrieval with **sparse lexical search (BM25 or SPLADE)**.



Reciprocal Rank Fusion (RRF)

RRF merges heterogeneous ranking lists by focusing on relative rank positions rather than comparing uncalibrated raw score distributions:

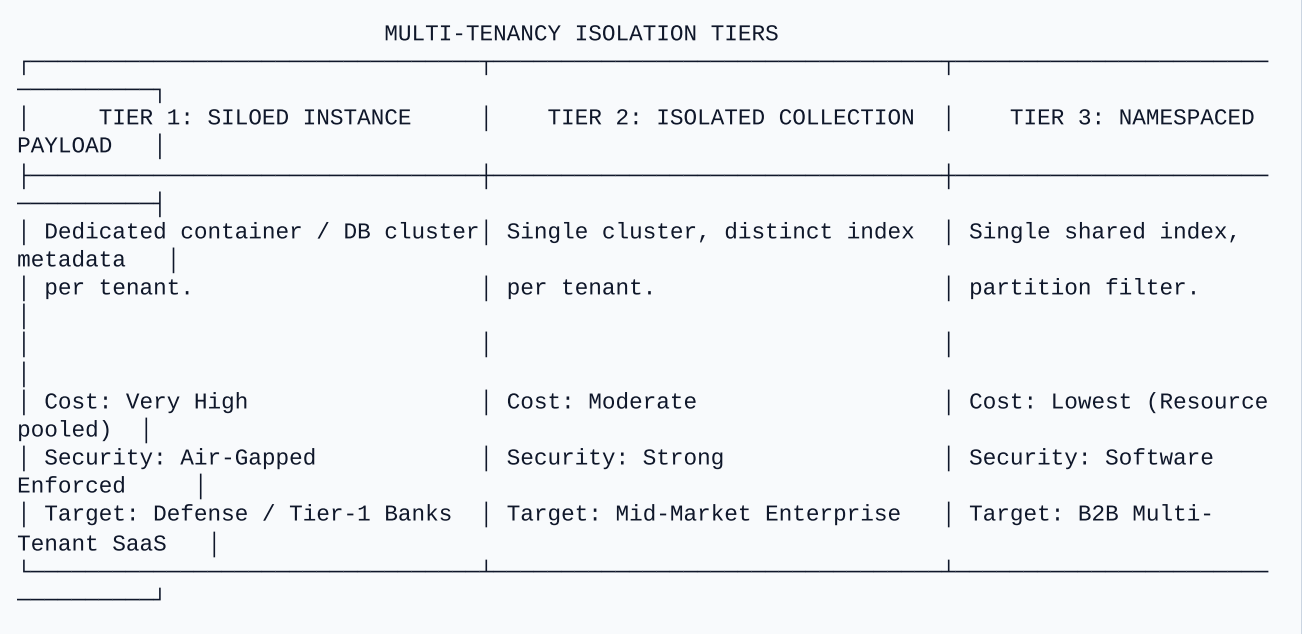
$$\text{RRF_Score}(d \in D) = \sum_{m \in M} \frac{1}{k + r_m(d)}$$

where:

- M is the set of retrieval systems (e.g., $M = \{\text{Dense}, \text{BM25}\}$).
- $r_m(d)$ is the ordinal rank position of document d in system m (starting at 1).
- k is a smoothing constant (typically $k = 60$) that prevents top-ranked items from dominating the fused score.

3. Multi-Tenant Vector Store Architectures

In SaaS enterprise environments, data isolation between distinct client accounts is a hard security and regulatory requirement. Vector databases support three multi-tenancy models:



1. **Siloed Infrastructure (Instance-per-Tenant):** Every tenant receives an isolated database container or cluster. Guarantees complete compute, memory, and cryptographic isolation, but incurs high infrastructure overhead and idle resource costs for small tenants.
2. **Collection-per-Tenant:** A single vector database cluster hosts dedicated collections/tables per tenant. Provides strong logical boundary isolation, but creates index memory overhead if the number of tenants scales into thousands (as each HNSW graph allocates fixed metadata and graph structures).
3. **Partitioned Shared Collection (Payload Filtering):** All tenants share a unified vector space, with vectors tagged using a `tenant_id` metadata attribute. Queries enforce a mandatory tenant constraint:

   $$\text{Filter} = \{ \texttt{"tenant_id"} : \texttt{"tenant_9821a"} \}$$

4. ◦ **Production Security Rule:** Tenant filters must be injected by backend middleware based on verified authorization tokens (JWT claims), never passed as unverified arguments from frontend clients.

Technical Summary of Module 9 Part 1

Component	Mathematical / Algorithmic Core	Primary Failure Mode	Production Best Practice
9.1 Dense Embeddings	Bi-Encoder contrastive InfoNCE loss; Mean/CLS pooling	Representation collapse; domain drift on specialized jargon	Match training pooling strategy; use task prefixes (e.g., Cohere `search_document`)
9.2 Similarity Metrics	L_2 Normalization: $\|\hat{\boldsymbol{x}}\|_2 = 1.0 \implies \|\hat{\boldsymbol{u}} - \hat{\boldsymbol{v}}\|_2^2 = 2(1 - \boldsymbol{u} \cdot \boldsymbol{v})$	Length bias on unnormalized dot products; compute bottlenecks	Unit-normalize vectors during ingestion; configure index for **Inner Product (IP)**
9.3 Chunking Strategies	Semantic distance shifts via dynamic thresholding: $\tau = \mu_d + k\sigma_d$	Truncating numbers/entities; context-free chunks	Use Recursive splitting for text; Semantic or Markdown Hierarchy for rich documents
9.4 Dimensionality & MRL	Multi-scale loss $\sum w_m L_m$; front-loaded semantics	Memory exhaustion (10^7 FP32 vectors $\approx 120 \text{ GB}$)	Two-stage retrieval: 256-dim MRL candidate shortlisting → full-dim reranking
9.5 Vector DB Landscape	Dedicated vector engines vs Relational vector extensions	Over-engineering architecture for modest dataset sizes	Default to `pgvector` for $<10 \text{M}$ records; scale to Qdrant/Milvus for dedicated needs
9.6 Indexing Algorithms	HNSW (Skip-List graphs); IVF (Centroid Voronoi cells); PQ (Subspace coding)	Memory overhead in HNSW ($1.75\times$); quantization loss	Use HNSW for low latency; pair with SQ8/PQ or MRL for memory efficiency
9.7 Metadata & Multi-Tenancy	Single-Stage Filtered HNSW; Reciprocal Rank Fusion: $\sum 1 / (k + r_m(d))$	Catastrophic Recall Collapse via naive post-filtering; data leakage	Single-stage filtered graphs; enforce server-side verified JWT tenant boundaries